Laboratório de Concorrência · Aula 10

Lucas Gabriel Godoy de Oliveira · RA 24214463-2 · Turma ESOFT6S-C
Gerado em 09/10/2026, 21:03:20 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização é um mecanismo que controla a ordem na qual as tarefas são executadas. Dois tipos de sincronização são necessários quando as tarefas compartilham dados: cooperação e competição. A sincronização de cooperação é necessária entre as tarefas A e B quando A deve esperar que B conclua alguma tarefa específica antes que possa começar ou continuar sua execução. A sincronização de competição é necessária entre duas tarefas quando ambas exigem o uso de algum recurso que não pode ser simultaneamente utilizado. Sincronização: músicos tocando no mesmo ritmo. Cooperação: amigos limpando juntos uma casa. Competição: dois atletas disputando uma corrida.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O valor esperado é 2.000.000, porque cada uma das threads incrementa o contador um milhão de vezes.

1ª execução (n = 1.000.000)

1486451

2ª execução (n = 1.000.000)

1012302

3ª execução (n = 1.000.000)

1307526

Saída do programa

```
esperado: 2000000
obtido:   1307526
```




```java
import java.util.*;

// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0;                // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;                  // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start();             // as duas executam ao mesmo tempo
        t1.join();  t2.join();              // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido:   " + contador);
    }
}
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores obtidos não foram iguais ao valor esperado de 2.000.000. Não foram iguais nas três execuções, porque as duas threads acessam e modificam o mesmo contador simultaneamente. Uma thread pode sobrescrever o resultado da outra, causando a perda de incrementos. A ordem de execução das threads pode mudar a cada execução, produzindo resultados diferentes.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Imagine que o contador vale 10 e que as duas threads tentam incrementá-lo ao mesmo tempo. Ler: a Thread 1 lê o valor 10; antes de gravar, a Thread 2 também lê 10. Somar: a Thread 1 calcula 10 + 1 = 11; a Thread 2 também calcula 10 + 1 = 11. Gravar: a Thread 1 grava 11; em seguida, a Thread 2 grava 11 também. O contador termina em 11, embora duas operações de incremento tenham sido realizadas. O correto seria 12. Isso acontece porque a segunda gravação sobrescreve a primeira, fazendo com que uma atualização seja perdida. Quando várias threads executam esse tipo de operação sem sincronização, o resultado final pode ficar abaixo do esperado.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o resultado foi 2000. Esse erro é perigoso em sistemas reais porque pode não aparecer durante os testes, mas ocorrer em produção, quando há mais threads e acessos simultâneos. Em sistemas bancários, por exemplo, uma atualização perdida poderia causar inconsistências em saldos ou registros.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABBAB → 6

ABBBAA → 4

BAAABB → 6

BBBAAA → 7

ABAABB → 6

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

main.py

```python
# Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
# Cole em https://onecompiler.com/python
# TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
# Cada tarefa tem 3 passos: 1) ler TOTAL  2) calcular  3) gravar TOTAL.
from itertools import combinations


def executa(ordem):                     # ordem: por exemplo "AABABB"
    total = 3
    reg = {"A": 0, "B": 0}              # o "registrador" de cada tarefa
    passo = {"A": 0, "B": 0}
    for t in ordem:
        if passo[t] == 0:
            reg[t] = total                                  # 1) ler
        elif passo[t] == 1:
            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2   # 2) calcular
        else:
            total = reg[t]                                  # 3) gravar
        passo[t] += 1
    return total


print("AAABBB ->", executa("AAABBB"))   # A inteira, depois B inteira
print("BBBAAA ->", executa("BBBAAA"))   # B inteira, depois A inteira
print("ABABAB ->", executa("ABABAB"))   # alternando passo a passo

resultados = {}
for posicoes in combinations(range(6), 3):        # onde ficam os 3 passos de A
    ordem = "".join("A" if i in posicoes else "B" for i in range(6))
    resultados.setdefault(executa(ordem), []).append(ordem)

print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
```

Console I/O

```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens. 4 resultados diferentes. Somente uma ordem dá o resultado correto: AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Uma ordem que produz 4 é AABBBA.

- 1: A lê: A lê 3 para seu registrador.

- 2: A calcula: A calcula 3 + 1 = 4, mas ainda não grava o resultado.
- 3: B lê: B lê o valor 3 de TOTAL, pois A ainda não gravou.
- 4: B calcula: B calcula 3 × 2 = 6.
- 5: B grava: B grava 6 em TOTAL.
- 6: A grava: A grava 4 em TOTAL, sobrescrevendo o 6.

Resultado final: TOTAL = 4.

A operação de B foi perdida porque A gravou seu resultado depois de B. Mesmo que B tenha calculado e gravado 6, a gravação posterior de A sobrescreveu esse valor. Isso é um exemplo de condição de corrida. O resultado depende da ordem em que as tarefas executam seus passos.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão com a operação atômica será a mais rápida, pois realiza o incremento sem precisar bloquear explicitamente uma seção crítica.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1446777 em 2 ms
synchronized:      2000000 em 189 ms
AtomicInteger:     2000000 em 26 ms
```

Main.java

```java
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

Console I/O

```
esperado:
2000000
sem sincronizacao:
1446777 em 2 ms
synchronized:
2000000 em 189 ms
AtomicInteger:
2000000 em 26 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões com synchronized e AtomicInteger apresentaram o valor correto de 2.000.000. A versão sem sincronização costuma ser mais rápida porque não precisa coordenar o acesso simultâneo ao contador. Já os mecanismos de sincronização acrescentam custos para garantir que as atualizações sejam realizadas corretamente.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo garante que as operações individuais sobre cada saldo sejam atômicas, mas não garante que a transferência inteira seja executada como uma única operação. Por exemplo, uma transferência de R$ 100 precisa realizar duas operações: retirar R$ 100 da conta de origem e adicionar R$ 100 à conta de destino. Se outra thread acessar as contas entre essas operações, poderá observar um estado intermediário inconsistente. Além disso, a retirada precisa verificar se há saldo suficiente. Para resolver isso, seria necessário proteger a transferência inteira como uma única seção crítica, usando um mecanismo de sincronização adequado, como synchronized, protegendo a retirada, a validação do saldo e o depósito como uma única operação indivisível. Em sistemas bancários reais, também são necessários mecanismos transacionais e de persistência para garantir a consistência dos dados.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens "A terminou" e "B terminou" provavelmente não vão aparecer, porque ocorre um deadlock. A thread A adquire o recurso X e depois tenta adquirir Y, que está bloqueado pela thread B. A thread B adquire o recurso Y e depois tenta adquirir X, que está bloqueado pela thread A. As duas ficam esperando indefinidamente, sem conseguir continuar.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas
-- -- -- -- -- -- -- -- -- - --
A terminou
B terminou
sem impasse
```

Main.java

```java
// Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
// Cole em https://onecompiler.com/java
import java.lang.management.ManagementFactory;

public class Main {
    static final Object X = new Object(), Y = new Object();   // dois recursos

    static void pausa() {
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }

    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> {
            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
        });
        Thread b = new Thread(() -> {
            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
        });
        a.start(); b.start();
        Thread.sleep(1000);                                    // espera 1 segundo
        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(presas == null ? "sem impasse"
                                          : "IMPASSE: " + presas.length + " threads presas");
        System.exit(0);                                        // encerra o programa
    }
}
```

Console

```
IMPASSE: 2
threads
presas
```

Main.java

```java
// Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
// Cole em https://onecompiler.com/java
import java.lang.management.ManagementFactory;

public class Main {
    static final Object X = new Object(), Y = new Object();   // dois recursos

    static void pausa() {
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }

    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> {
            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
        });
        Thread b = new Thread(() -> {
            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
        });
        a.start(); b.start();
        Thread.sleep(1000);                                    // espera 1 segundo
        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(presas == null ? "sem impasse"
                                          : "IMPASSE: " + presas.length + " threads presas");
        System.exit(0);                                        // encerra o programa
    }
}
```

Console

```
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque cada uma segurou um recurso de que a outra precisava para continuar. Thread A: adquiriu o recurso X e depois tentou adquirir Y, mas Y estava bloqueado pela thread B. Thread B: adquiriu

o recurso Y e depois tentou adquirir X, mas X estava bloqueado pela thread A. Assim, A ficou esperando Y, enquanto B ficou esperando X. Nenhuma delas consegue liberar o recurso que já possui.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Se todas as threads adquirirem os recursos sempre na mesma ordem, por exemplo, primeiro X e depois Y, uma thread não poderá segurar Y enquanto espera por X. Dessa forma, não causa o impasse. Exemplo real: em um sistema bancário, duas transferências simultâneas podem precisar bloquear duas contas. Uma transferência tenta bloquear primeiro a conta A e depois a B, enquanto outra tenta bloquear primeiro a B e depois a A. Se cada uma bloquear uma conta, ambas podem ficar esperando indefinidamente.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Cada tarefa leva 100 ms, e há 6 tarefas no total. É esperado os seguintes tempos:
3 trabalhadores: aproximadamente 200 ms, pois cada trabalhador executa 2 tarefas.
1 trabalhador: aproximadamente 600 ms, pois executa as 6 tarefas uma após a outra.
6 trabalhadores: aproximadamente 100 ms, pois cada trabalhador executa uma tarefa ao mesmo tempo.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

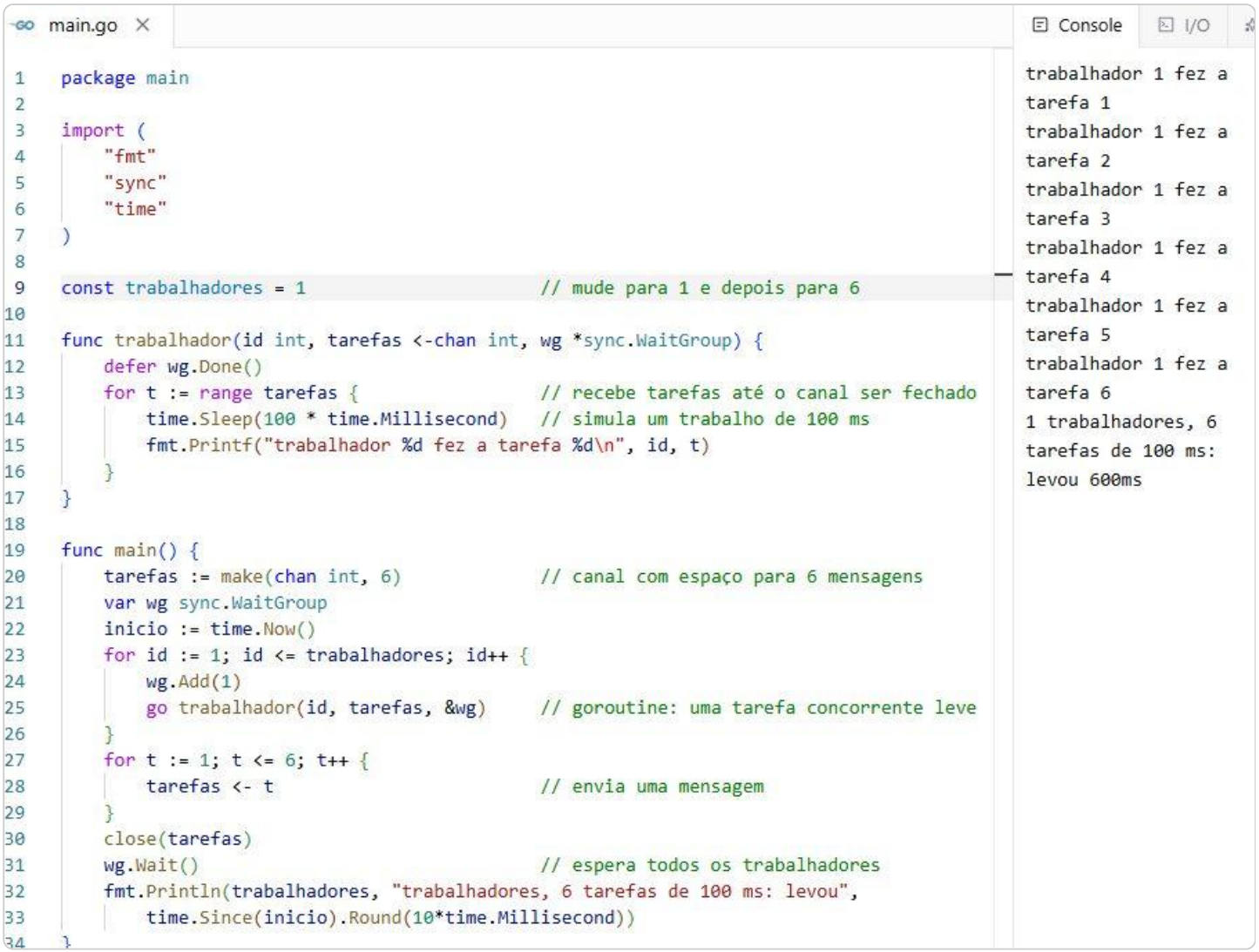

package main

import (
    "fmt"
    "sync"
```

```
    "time"
)

const trabalhadores = 6                  // mude para 1 e depois para 6


407 ms
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
trabalhador 5 fez a tarefa 6
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```



main.go

```go
package main

import (
    "fmt"
    "sync"
    "time"
)

const trabalhadores = 3                    // mude para 1 e depois para 6

func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for t := range tarefas {               // recebe tarefas até o canal ser fechado
        time.Sleep(100 * time.Millisecond)  // simula um trabalho de 100 ms
        fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
    }
}

func main() {
    tarefas := make(chan int, 6)           // canal com espaço para 6 mensagens
    var wg sync.WaitGroup
    inicio := time.Now()
    for id := 1; id <= trabalhadores; id++ {
        wg.Add(1)
        go trabalhador(id, tarefas, &wg)   // goroutine: uma tarefa concorrente leve
    }
    for t := 1; t <= 6; t++ {
        tarefas <- t                       // envia uma mensagem
    }
    close(tarefas)
    wg.Wait()                              // espera todos os trabalhadores
    fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
        time.Since(inicio).Round(10*time.Millisecond))
}
```

Console I/O

```
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 3 fez a tarefa 6
trabalhador 1 fez a tarefa 4
trabalhador 2 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

main.go

```go
package main

import (
    "fmt"
    "sync"
    "time"
)

const trabalhadores = 6                  // mude para 1 e depois para 6

func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for t := range tarefas {                // recebe tarefas até o canal ser fechado
        time.Sleep(100 * time.Millisecond)   // simula um trabalho de 100 ms
        fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
    }
}

func main() {
    tarefas := make(chan int, 6)             // canal com espaço para 6 mensagens
    var wg sync.WaitGroup
    inicio := time.Now()
    for id := 1; id <= trabalhadores; id++ {
        wg.Add(1)
        go trabalhador(id, tarefas, &wg)     // goroutine: uma tarefa concorrente leve
    }
    for t := 1; t <= 6; t++ {
        tarefas <- t                         // envia uma mensagem
    }
    close(tarefas)
    wg.Wait()                                // espera todos os trabalhadores
    fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
        time.Since(inicio).Round(10*time.Millisecond))
}
```

Console I/O

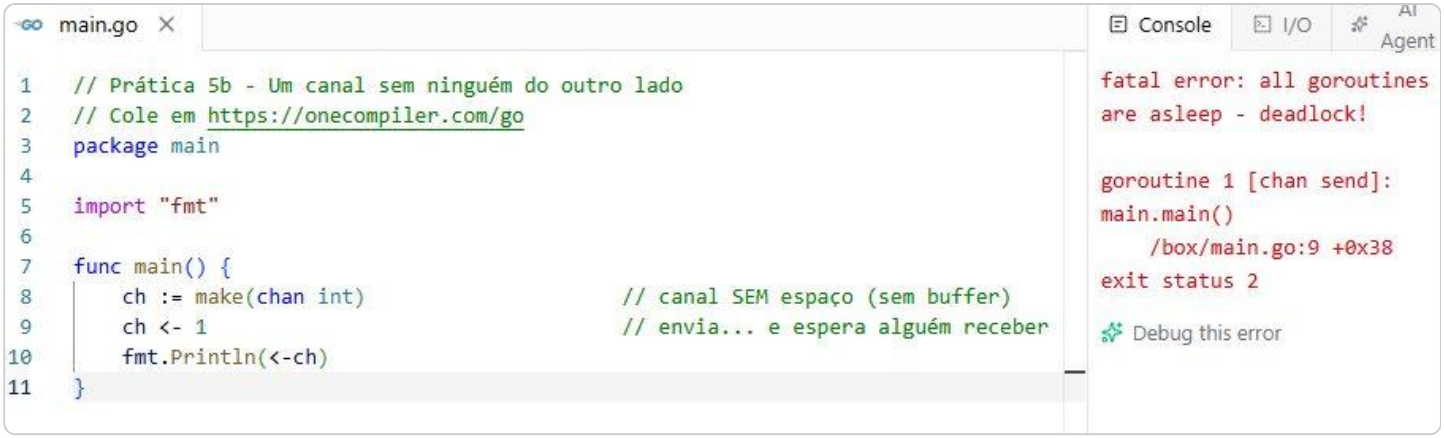
```
trabalhador 2 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 3 fez a tarefa 3
trabalhador 4 fez a tarefa 4
trabalhador 5 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

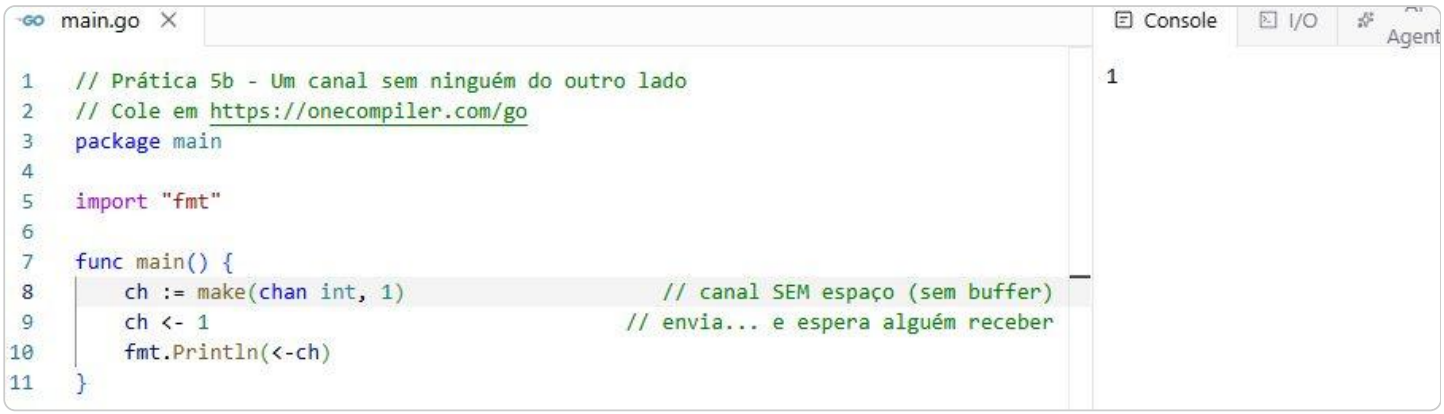
5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

1 trabalhador: aproximadamente 600 ms, pois executa as seis tarefas sequencialmente, cada uma levando 100 ms.
3 trabalhadores: aproximadamente 200 ms, pois cada trabalhador executa duas tarefas em paralelo com os demais.
6 trabalhadores: aproximadamente 100 ms, pois cada trabalhador executa uma tarefa ao mesmo tempo.
O programa não precisa de synchronized nem de travas porque as tarefas são distribuídas por um canal, que coordena o envio e o recebimento das mensagens. Os trabalhadores recebem uma tarefa por vez e executam o trabalho sem precisar compartilhar e modificar um contador comum.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!
goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2
- - - - - - - - - - - - - - - - - -
1
```





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque make(chan int) cria um canal sem buffer. Quando o programa executa ch <- 1, tenta enviar um valor, mas não existe nenhuma goroutine recebendo esse valor. O envio fica bloqueado à espera de um receptor. Como a execução fica parada nessa linha, o programa nunca chega ao fmt.Println(<-ch).

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa não vai compilar. Em Rust, a thread criada com thread::spawn precisa ter acesso seguro aos dados que utiliza. Como a variável pertence à função main, a nova thread não pode simplesmente alterá-la sem que as regras de propriedade e compartilhamento sejam respeitadas.

Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
 ⟶ HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |                           ^^ may outlive borrowed value `contador`
8 |         contador += 1;                       // a thread altera a variável de main
  |         -------- `contador` is borrowed here
  |
note: function requires argument type to outlive `'static`
 ⟶ HelloWorld.rs:7:13
  |
7 |       let t = thread::spawn(|| {
  |  _____________^
```

```
  8 | |         contador += 1;                       // a thread altera a variável de main
  9 | |     });
    | |______^
  help: to force the closure to take ownership of `contador` (and any other referenced
  variables), use the `move` keyword
    |
  7 |     let t = thread::spawn(move || {
    |                           ++++

  error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
    --> HelloWorld.rs:11:20
     |
   7 |       let t = thread::spawn(|| {
     |               -             -- mutable borrow occurs here
     |  _____________|
     | |
   8 | |         contador += 1;                       // a thread altera a variável de main
     | |         -------- first borrow occurs due to use of `contador` in closure
   9 | |     });
     | |______- argument requires that `contador` is borrowed for `'static`
  10 |       t.join().unwrap();
  11 |       println!("{}", contador);
     |                      ^^^^^^^^ immutable borrow occurs here
     |
  note: requirement that the value outlives `'static` introduced here
    -->
  /rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14

  error: aborting due to 2 previous errors

  Some errors have detailed explanations: E0373, E0502.
  For more information about an error, try `rustc --explain E0373`.


  warning: value captured by `contador` is never read
   --> HelloWorld.rs:8:9
    |
  8 |         contador += 1;                       // a thread altera a variável de main
    |         ^^^^^^^^
    |
    = help: did you mean to capture by reference instead?
    = note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

  warning: value assigned to `contador` is never read
   --> HelloWorld.rs:8:9
    |
  8 |         contador += 1;                       // a thread altera a variável de main
    |         ^^^^^^^^^^^^
    |
    = help: maybe it is overwritten before being read?

  warning: 2 warnings emitted
```

HelloWorld.rs

```rust
// Prática 6a - Rust: tentando compartilhar uma variável entre threads
// Cole em https://onecompiler.com/rust
use std::thread;

fn main() {
    let mut contador = 0;
    let t = thread::spawn(|| {
        contador += 1;                    // a thread altera a variável de main
    });
    t.join().unwrap();
    println!("{}", contador);
}
```

Console | I/O | AI Agent

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
 --> HelloWorld.rs:7:27
  |
7 |     let t = thread::spawn(|| {
  |
^^ may outlive borrowed value `contador`
8 |         contador += 1;
// a thread altera a variável de main
  |         --------
`contador` is borrowed here
  |
note: function requires argument type to outlive `'static`
 --> HelloWorld.rs:7:13
  |
7 |       let t = thread::spawn(|| {
  |  _____________^
8 | |         contador += 1;
// a thread altera a variável de main
9 | |     });
  | |_____^
help: to force the closure to take ownership of `contador` (and any other referenced
```

HelloWorld.rs

```rust
// Prática 6a - Rust: tentando compartilhar uma variável entre threads
// Cole em https://onecompiler.com/rust
use std::thread;

fn main() {
    let mut contador = 0;
    let t = thread::spawn(move || {
        contador += 1;                    // a thread altera a variável de main
    });
    t.join().unwrap();
    println!("{}", contador);
}
```

Console I/O AI Agent

```
warning: value captured by `contador` is never read
 --> HelloWorld.rs:8:9
  |
8 |         contador += 1; // a thread altera a variável de main
  |         ^^^^^^^^
  |
  = help: did you mean to capture by reference instead?
  = note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
 --> HelloWorld.rs:8:9
  |
8 |         contador += 1; // a thread altera a variável de main
  |         ^^^^^^^^^^^^
  |
  = help: maybe it is overwritten before being read?

warning: 2 warnings emitted

0
```

Saída do programa

```
contador = 400000
```



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread recebe a propriedade de uma cópia do valor da variável contador, em vez de alterar a variável original de main. Assim, a thread incrementa sua própria cópia, enquanto o contador original permanece em 0. Por isso, o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a condição de corrida da Prática 1 só foi percebida durante a execução, quando o contador apresentou um resultado menor que o esperado. O programa compilou normalmente, mesmo contendo o problema. Em Rust, o primeiro programa é rejeitado pelo compilador porque a thread tenta acessar e modificar uma variável de maneira que não respeita as regras de propriedade e segurança entre threads. A vantagem do Rust é detectar muitos problemas de concorrência antes da execução, reduzindo a possibilidade de erros difíceis de reproduzir em produção. O custo para quem programa é precisar compreender e aplicar corretamente conceitos como propriedade, empréstimos, Arc e Mutex, o que pode tornar o código inicial mais trabalhoso.

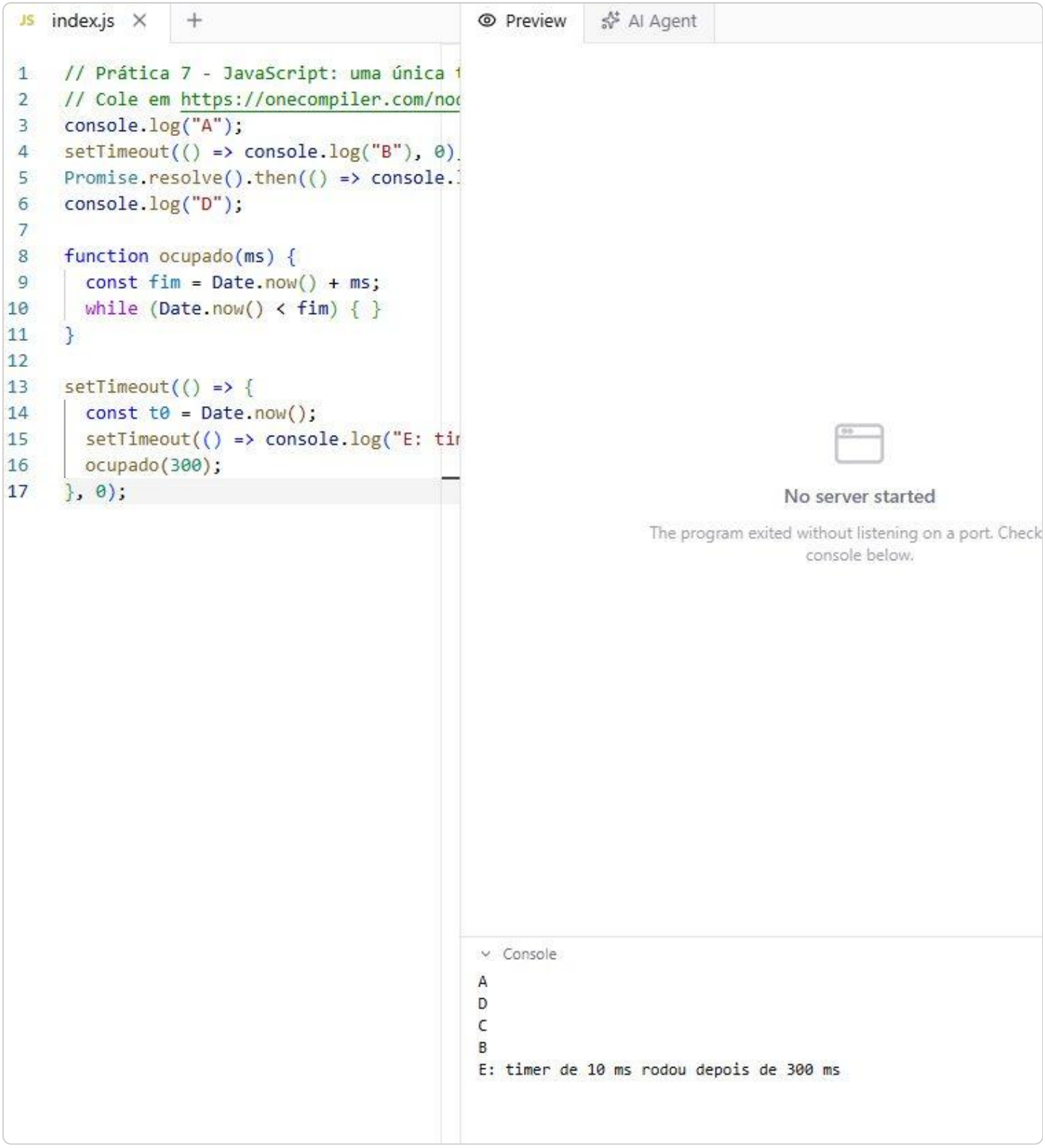
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B, E. Porque setTimeout(..., 0) não executa a função imediatamente: ele agenda o callback para depois que o código síncrono terminar. D sai primeiro porque faz parte do código síncrono. C sai em seguida porque é uma microtarefa de Promise, processada antes dos temporizadores. B sai depois porque é um callback de setTimeout, processado na fila de tarefas do laço de eventos.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O temporizador de 10 ms foi executado mais tarde porque o laço ocupado(300) bloqueou a thread principal por aproximadamente 300 ms. Durante esse período, o Node.js não conseguiu processar outros callbacks nessa thread,

mesmo que seus temporizadores já estivessem prontos. Em um servidor Node.js que atende muitos usuários, um cálculo pesado executado diretamente na thread principal pode atrasar o processamento de todas as outras requisições. Isso aumenta a latência, prejudica a capacidade de resposta e pode fazer com que os usuários esperem mais tempo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida. As duas pessoas podem comprar o último ingresso porque ambas verificam que ainda há um lugar disponível antes de qualquer uma concluir a compra. Para evitar isso, usaria sincronização no servidor, com uma operação atômica ou transação no banco de dados que verifique a disponibilidade e reserve o ingresso como uma única operação.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que um programa pode funcionar corretamente em alguns testes e apresentar erros em outras execuções por causa da concorrência entre tarefas.